DAY 7 · EKS VS AKS VS GKE

One Kubernetes skill set.
Three managed services.
A dozen differences that bite.



INFOGRAPHIC 1/3 MANAGED KUBERNETES ACROSS CLOUDS

Running Kubernetes on EKS, AKS and GKE: the decisions, in order

The Kubernetes API is the same everywhere. The provider manages the control plane; everything around it (nodes, pod networking, identity, access, upgrades) is where the three services differ, and where teams get caught.

01

Choose the operating model

Who manages the nodes: you (node groups / node pools), or the provider (EKS Auto Mode or Fargate, GKE Autopilot).

Why: it decides how much patching, scaling and capacity planning stays with your team.

```
gcloud container clusters create-auto demo --region us-central1   # GKE Autopilot
```

02

Plan pod networking and IP space

EKS's VPC CNI gives pods VPC IPs; AKS commonly uses Azure CNI Overlay; GKE clusters are VPC-native with alias IP ranges.

Why: pod IP exhaustion and overlapping ranges are hard to fix after the cluster exists.

```
az aks create -g rg-demo -n demo --network-plugin azure --network-plugin-mode overlay --pod-cidr 192.168.0.0/16
```

03

Decide who can reach the cluster

Map cloud identities to Kubernetes RBAC: EKS access entries, AKS with Microsoft Entra ID, GKE with IAM plus RBAC.

Why: the cloud console role and the in-cluster role are separate; both need least privilege.

```
aws eks create-access-entry --cluster-name demo --principal-arn arn:aws:iam::111122223333:role/platform-admins
```

04

Give pods cloud access without keys

EKS Pod Identity (or IRSA), Microsoft Entra Workload ID, Workload Identity Federation for GKE.

Why: keys mounted as secrets leak and never rotate; federated identity is short-lived.

```
aws eks create-pod-identity-association --cluster-name demo --namespace app \
  --service-account app-sa --role-arn arn:aws:iam::111122223333:role/app-role
```

INFOGRAPHIC 2/3 TRAFFIC, DATA, SIGNALS, UPGRADES

Then wire up what every workload depends on

05

Expose services through the provider's load balancers

AWS Load Balancer Controller (ALB/NLB), AKS application routing or Application Gateway for Containers, GKE Ingress or Gateway.

Why: annotations and controllers differ per cloud; keep them out of the app's base manifests.

```
kubectl get gatewayclass   # see which Gateway API classes the cluster offers
```

06

Choose storage classes deliberately

Block storage comes through CSI drivers: EBS on EKS (installed as an add-on), Azure Disk on AKS, Persistent Disk on GKE.

Why: disks are zonal; a pod rescheduled to another zone can't attach its volume.

```
kubectl get storageclass   # check provisioner, binding mode and reclaim policy
```

07

Turn on metrics and logs before the first incident

Container Insights on AWS, Azure Monitor with managed Prometheus, Cloud Monitoring and Managed Service for Prometheus on GKE.

Why: the same dashboards and alerts on every cluster make cross-cloud on-call possible.

```
kubectl top nodes && kubectl get events -A --sort-by=.lastTimestamp | tail
```

08

Own the upgrade cadence

Minor versions upgrade one at a time: control plane first, then nodes and add-ons. Use maintenance windows.

Why: versions leave standard support on a fixed schedule; falling behind means forced or paid upgrades.

```
kubectl get --raw /metrics | grep apiserver_requested_deprecated_apis   # find APIs to fix first
```

	EKS	AKS	GKE
Kubeconfig	aws eks update-kubeconfig	az aks get-credentials	gcloud container clusters get-credentials
Upgrade model	You trigger; Auto Mode manages nodes	Auto-upgrade channels, planned maintenance	Release channels, maintenance windows and exclusions

INFOGRAPHIC 3/3 SIDE BY SIDE

EKS vs AKS vs GKE: what actually differs

	EKS	AKS	GKE
Hands-off nodes	EKS Auto Mode, Fargate	Virtual nodes (ACI)	Autopilot
Your nodes	Managed node groups, Karpenter	System and user node pools	Standard node pools, auto-provisioning
Pod networking	VPC CNI: pods use VPC IPs	Azure CNI Overlay or Azure CNI	VPC-native alias IPs, Dataplane V2
Cluster access	Access entries (replace aws-auth)	Entra ID + Azure RBAC or K8s RBAC	IAM + Kubernetes RBAC
Pod identity	EKS Pod Identity, IRSA	Entra Workload ID	Workload Identity Federation
Load balancing	AWS Load Balancer Controller	App routing, App Gateway for Containers	GKE Ingress, Gateway
Block storage	EBS CSI driver (add-on)	Azure Disk CSI	Persistent Disk CSI
Managed metrics	Container Insights, Managed Prometheus	Container insights, managed Prometheus	Cloud Monitoring, Managed Prometheus
Control plane	Per-cluster fee; paid extended support	Free tier, or Standard tier with SLA	Per-cluster fee

✓ Must know

- **The control plane is managed;** nodes, add-ons and workloads are still yours to secure and patch
- **Keep base manifests cloud-neutral;** put cloud annotations in per-cloud overlays
- **Pod identity, never keys** in Kubernetes secrets
- **One minor version at a time**; read the deprecation notes first
- **Private API endpoint** where your network allows it

✗ Pitfalls

- Running out of pod IPs in a small subnet
- Cloud credentials stored as Kubernetes secrets
- Falling behind on versions until support ends
- Zonal disks with workloads spread across zones
- Cluster-admin for everyone "just for now"

WHAT & WHY

Managed Kubernetes: what the provider runs, and what you still own

The provider runs

The control plane: API server, etcd, scheduler and controllers, their availability and their patching. You never SSH into it.

You still own

Node choice and patching (unless a hands-off mode does it), pod networking, who can reach the cluster, workload identity, add-ons, upgrades and everything you deploy.

What stays the same

kubectl, Deployments, Services, RBAC, Helm and your manifests. The Kubernetes API is certified conformant on all three.

Why it matters: teams that move between EKS, AKS and GKE rarely struggle with Kubernetes itself. They struggle with the provider-specific layer around it.

THE MAP

8 decisions for every managed cluster

01 **Operating model:** who manages nodes, you or the provider

02 **Pod networking:** CNI mode and IP ranges, planned before creation

03 **Cluster access:** cloud identities mapped to Kubernetes RBAC

04 **Workload identity:** pods reach cloud APIs without stored keys

05 **Traffic:** ingress or Gateway through the provider's load balancers

06 **Storage:** CSI drivers, storage classes and zones

07 **Signals:** metrics, logs and alerts from day one

08 **Upgrades:** a version cadence you own, before support ends

> **Exercise:** answer these eight for a cluster you run today. Any "not sure" is your next task.

DECISION 1 · OPERATING MODEL

How much of the node layer do you want?

	You manage nodes	Provider manages nodes
EKS	Managed node groups, self-managed nodes, Karpenter	EKS Auto Mode; Fargate per pod
AKS	System and user node pools, cluster autoscaler	Node auto-provisioning; virtual nodes on ACI
GKE	Standard mode node pools, node auto-provisioning	Autopilot (billed per pod resource requests)

→ **Hands-off modes** trade control (custom node images, privileged DaemonSets) for less patching and capacity work

→ **Your own nodes** fit GPU, special kernels or strict cost tuning

→ **Mix where allowed:** a small system pool for add-ons, flexible pools for workloads

> **Start simple:** for a first cluster, a hands-off mode removes most of the day-2 node work.

DECISION 2 · POD NETWORKING

Plan IP space before the cluster exists

EKS
Amazon VPC CNI: every pod gets a VPC IP. Simple routing, but subnets fill fast. Prefix delegation helps.

AKS
Azure CNI Overlay: pods use a private CIDR separate from the VNet, so VNet IPs are saved.

GKE
VPC-native clusters: pods and Services use secondary (alias) IP ranges on the subnet.

→ **Size for growth:** nodes × max pods per node, plus room for upgrade surge nodes

→ **No overlaps** with on-premises, peered networks or other clusters you'll connect

→ **Network policy:** decide the engine early (Calico, Cilium, or the provider's) and default-deny per namespace

> **Hard to undo:** changing a cluster's pod networking usually means building a new cluster and migrating.

DECISION 3 · CLUSTER ACCESS

Two layers of permission, both least privilege

- → **Cloud layer:** who can call the provider's API to get credentials or change the cluster
- → **Kubernetes layer:** what they can do inside, through RBAC roles and bindings
- → **EKS:** access entries with access policies (they replace editing the aws-auth ConfigMap)
- → **AKS:** Microsoft Entra ID sign-in, with Azure RBAC or Kubernetes RBAC for authorization
- → **GKE:** IAM roles for the project, Kubernetes RBAC inside; Google Groups can be bound in RBAC

```
# EKS: give a team role read-only access to one namespace
aws eks create-access-entry --cluster-name demo \
  --principal-arn arn:aws:iam::111122223333:role/dev-team
aws eks associate-access-policy --cluster-name demo \
  --principal-arn arn:aws:iam::111122223333:role/dev-team \
  --policy-arn arn:aws:eks::aws:cluster-access-policy/AmazonEKSViewPolicy
\
  --access-scope type=namespace,namespaces=app
```

DECISION 4 · WORKLOAD IDENTITY

Pods get short-lived cloud credentials, not keys

The pattern is the same everywhere: a Kubernetes service account is trusted by a cloud identity.

```
# EKS Pod Identity (needs the eks-pod-identity-agent add-on)
aws eks create-pod-identity-association --cluster-name demo \
  --namespace app --service-account app-sa --role-arn "$ROLE_ARN"

# AKS: Microsoft Entra Workload ID
az aks update -g rg-demo -n demo --enable-oidc-issuer --enable-workload-
identity
az identity federated-credential create --name app-fic \
  --identity-name app-id -g rg-demo --issuer "$AKS_OIDC_ISSUER" \
  --subject system:serviceaccount:app:app-sa

# GKE: Workload Identity Federation (on by default in Autopilot)
POOL="projects/$PROJECT_NUMBER/locations/global/workloadIdentityPools/$PRO
JECT_ID.svc.id.goog"
gcloud storage buckets add-iam-policy-binding gs://my-bucket \
  --role=roles/storage.objectViewer \
  --member="principal://iam.googleapis.com/$POOL/subject/ns/app/sa/app-sa"
```

AKS detail: annotate the service account with the identity's client ID and label the pod `azure.workload.identity/use: "true"`.

DECISION 5 · TRAFFIC

Keep the cloud-specific parts in overlays

	Layer 4 (Service type LoadBalancer)	Layer 7
EKS	NLB via AWS Load Balancer Controller	ALB via Ingress (controller installed by you)
AKS	Azure Load Balancer	Application routing add-on, App Gateway for Containers
GKE	Passthrough Network Load Balancer	GKE Ingress or Gateway API

```
# One base, one small overlay per cloud (Kustomize)
k8s/
  base/                 # Deployment, Service, HPA: cloud-neutral
  overlays/
    eks/  aks/  gke/    # annotations, storage class, ingress class
kubectl apply -k k8s/overlays/gke
```

Gateway API is supported across the three in different forms; check which GatewayClass your cluster offers before you standardize on it.

DECISIONS 6 & 7 · STORAGE AND SIGNALS

Zonal disks and day-one observability

- → **CSI drivers:** EBS on EKS (install the add-on), Azure Disk and Azure Files on AKS, Persistent Disk and Filestore on GKE
- → **Use `WaitForFirstConsumer`** binding so the disk is created in the zone where the pod lands
- → **Set the reclaim policy** deliberately: Delete loses data with the claim, Retain leaves orphaned disks to clean up
- → **StatefulSets across zones** need one volume per replica in each zone

EKS

CloudWatch Container Insights, Amazon Managed Service for Prometheus

AKS

Azure Monitor Container insights, Azure Monitor managed Prometheus

GKE

Cloud Logging and Monitoring, Managed Service for Prometheus

Portable signals: Prometheus metrics and OpenTelemetry traces let one set of dashboards and alerts work on every cluster.

DECISION 8 · UPGRADES

A version cadence you own

01 **Read the release notes** and find removed APIs your manifests and Helm charts still use

02 **Upgrade the control plane** one minor version at a time

03 **Upgrade add-ons** (CNI, CSI, ingress, CoreDNS) to versions that support the new release

04 **Roll the nodes** with surge capacity and PodDisruptionBudgets so workloads stay up

05 **Verify,** then repeat on the next environment

```
# What still calls deprecated APIs?
kubectl get --raw /metrics | grep apiserver_requested_deprecated_apis
# Who controls the timing
EKS: you start upgrades; extended support costs more after standard
support ends
AKS: auto-upgrade channel + planned maintenance window
GKE: release channel + maintenance windows and exclusions
```

> **Kubernetes skew policy:** nodes may lag the control plane by a limited number of minor versions, never lead it.

COST

Where managed Kubernetes money goes

- → **Control plane:** EKS and GKE charge a per-cluster management fee; AKS has a Free tier and a paid Standard tier with an uptime SLA
- → **Nodes:** usually the biggest line. Right-size requests, use autoscaling, and Spot or preemptible capacity for tolerant workloads
- → **Hands-off modes** bill differently: GKE Autopilot by pod resource requests, EKS Auto Mode adds a management charge per instance
- → **Load balancers, NAT and cross-zone traffic** add up quietly
- → **Old versions:** EKS extended support carries a higher per-cluster rate
- → **Idle clusters:** one per developer adds up; prefer namespaces on a shared dev cluster

> **Always check** the current pricing page for your region before you choose; this slide shows where cost comes from, not the prices.

PORTABILITY

What to keep identical across clouds

Keep the same

Base manifests or Helm charts, RBAC roles, namespaces, resource requests, probes, PodDisruptionBudgets, network policies, dashboards and alerts.

Allow to differ

Node configuration, CNI settings, load balancer annotations, storage classes, identity wiring and the provider's add-ons.

→ **Infrastructure as code** for each cluster (Terraform modules per provider)

→ **GitOps** (Argo CD or Flux) applying the same base with a per-cloud overlay

→ **Policy as code** (Kyverno or OPA Gatekeeper) enforcing the same rules everywhere

Goal: a new engineer can read the base and know how the app runs, whichever cloud it's on.

PITFALLS

8 managed Kubernetes mistakes

- → **Small subnets:** pods can't schedule when IPs run out
- → **Cloud keys in Kubernetes secrets** instead of workload identity
- → **Cluster-admin for everyone**, "just for now"
- → **Skipping versions** until support ends and upgrades are forced
- → **Public API endpoint** open to the internet with no IP restrictions
- → **Cloud annotations** hard-coded in the app's base manifests
- → **No PodDisruptionBudgets**, so node upgrades take the app down
- → **No resource requests**, so autoscaling and bin-packing can't work

CHECKLIST

Managed Kubernetes checklist

- ☐ Operating model chosen and written down
- ☐ Pod and Service IP ranges sized, with no overlaps
- ☐ Cluster access through access entries, Entra ID or IAM; no shared admin
- ☐ Workload identity for every pod that calls a cloud API
- ☐ Cloud-specific settings live in overlays, not the base
- ☐ Storage classes with WaitForFirstConsumer and a deliberate reclaim policy
- ☐ Metrics, logs and alerts on before go-live
- ☐ Upgrade cadence, maintenance window and deprecated-API check
- ☐ PodDisruptionBudgets and resource requests on every workload
- ☐ API endpoint private or IP-restricted

HANDS-ON LAB

Try it: one app on two managed services

```
# GKE Autopilot
gcloud container clusters create-auto lab --region us-central1
gcloud container clusters get-credentials lab --region us-central1
# AKS
az group create -n rg-k8s-lab -l eastus
az aks create -g rg-k8s-lab -n lab --node-count 2 --generate-ssh-keys
az aks get-credentials -g rg-k8s-lab -n lab
# EKS (eksctl)
eksctl create cluster --name lab --region us-east-1 --nodes 2

# On each cluster: same app, then note every difference
kubectl create deployment web --image=nginx:1.27 --replicas=2
kubectl expose deployment web --port=80 --type=LoadBalancer
kubectl get svc web --watch
```

Teardown: `kubectl delete svc web` first (it frees the load balancer), then `gcloud container clusters delete lab --region us-central1`, `az group delete -n rg-k8s-lab --yes`, `eksctl delete cluster --name lab --region us-east-1`.

FURTHER READING

Go deeper: official references

AWS: What is Amazon EKS?
docs.aws.amazon.com/eks/latest/userguide/what-is-eks.html

Microsoft: What is Azure Kubernetes Service?
learn.microsoft.com/en-us/azure/aks/what-is-aks

Google Cloud: GKE overview
cloud.google.com/kubernetes-engine/docs/concepts/kubernetes-engine-overview

AWS: EKS Pod Identity
docs.aws.amazon.com/eks/latest/userguide/pod-identities.html

Kubernetes: Version skew policy
kubernetes.io/releases/version-skew-policy/

Docs change. Always check the current version before you design.

YOUR TURN

Which managed Kubernetes service would you learn first: EKS, AKS or GKE?

Tell me in the comments. I read and reply to every one.

Follow Deji Ajishe

Day 7 of my DevOps series

♻️ Repost to help an engineer on your team

linkedin.com/in/deji-ajishe-64a35722a